

Chapter 1. Optimization and Performance Defined

Optimizing the performance of Java (or any other sort of code) is often seen as a dark art. There's a mystique about performance analysis—it's



O'REILLY



puters and the people who operate them). The image is one of a single individual who can see deeply into a system and come up with a magic solution that makes the system work faster.

This image is often coupled with the unfortunate (but all-too-common) situation where performance is a second-class concern of the software teams. This sets up a scenario where analysis is only done once the system is already in trouble and needs a performance “hero” to save it. The reality, however, is a little different.

The truth is that performance analysis is a weird blend of hard empiricism and squishy human psychology. What matters is, at one and the same time, the absolute numbers of observable metrics and how the end users and stakeholders *feel* about them. The resolution of this apparent paradox is the subject of the rest of this book.

Since the publication of the first edition, this situation has only sharpened. As more and more workloads move into the cloud, and as systems become ever-more complicated, the strange brew that combines very different factors has become even more important and prevalent. The “domain of concern” that an engineer who cares about performance needs to operate in has continued to broaden.

This is because production systems have become even more complicated. More of them now have aspects of distributed systems to consider in addition to the performance of individual application processes. As system

architectures become larger and more complex, the number of engineers who must concern themselves with performance has also increased.

The new edition of this book responds to these changes in our industry by providing four things:

- A necessary deep-dive on the performance of application code running within a single Java Virtual Machine (JVM)
- A discussion of JVM internals
- Details of how the modern cloud stack interacts with Java/JVM applications
- A first look at the behavior of Java applications running on a cluster in a cloud environment

In this chapter, we will get going by setting the stage with some definitions and establishing a framework for *how* we talk about performance—starting with some problems and pitfalls that plague many discussions of Java performance.

Java Performance the Wrong Way

For many years, one of the top three hits on Google for “Java performance tuning” was an article from 1997–8, which had been ingested into the index very early in Google’s history. The page had presumably stayed close to the top because its initial ranking served to actively drive traffic to it, creating a feedback loop.

The page housed advice that was completely out of date, no longer true, and in many cases, detrimental to applications. However, its favored position in the search engine results caused many, many developers to be exposed to terrible advice.

For example, very early versions of Java had terrible method dispatch performance. As a workaround, some Java developers advocated avoiding small methods and instead writing monolithic methods. Of course, over time, the performance of virtual dispatch greatly improved.

Not only that, but with modern JVM technologies (especially automatic managed inlining), virtual dispatch has now been eliminated at a large number—perhaps even the majority—of call sites. Code that followed the

“lump everything into one method” advice is now at a substantial disadvantage, as it is very unfriendly to modern just-in-time (JIT) compilers.

There’s no way of knowing how much damage was done to the performance of applications that were subjected to the bad advice, but this case neatly demonstrates the dangers of not using a quantitative and verifiable approach to performance. It also provides yet another excellent example of why you shouldn’t believe everything you read on the internet.

NOTE

The execution speed of Java code is highly dynamic and fundamentally depends on the underlying Java virtual machine. An old piece of Java code may well execute faster on a more recent JVM, even without recompiling the Java source code.

As you might imagine, for this reason (and others we will discuss later) this book is not a cookbook of performance tips to apply to your code. Instead, we focus on a range of aspects that come together to produce good performance engineering:

- Performance methodology within the overall software lifecycle
- Theory of testing as applied to performance
- Measurement, statistics, and tooling
- Analysis skills (both systems and data)
- Underlying technology and mechanisms

By bringing these aspects together, the intention is to help you build an understanding that can be broadly applied to whatever performance circumstances you may face.

Later in the book, we will introduce some heuristics and code-level techniques for optimization, but these all come with caveats and tradeoffs that the developer should be aware of before using them.

TIP

Please do not skip ahead to those sections and start applying the techniques detailed without properly understanding the context in which the advice is given. All of these techniques are capable of doing more harm than good if you lack a proper understanding of how—and *why*—they should be applied.

In general, there are:

- No magic “go faster” switches for the JVM
- No “tips and tricks” to make Java run faster
- No secret algorithms that have been hidden from you

As we explore our subject, we will discuss these misconceptions in more detail, along with some other common mistakes that developers often make when approaching Java performance analysis and related issues.

Still here? Good. Then let’s talk about performance.

Java Performance Overview

To understand why Java performance is the way that it is, let’s start by considering a classic quote from James Gosling, the creator of Java:

*Java is a blue collar language. It’s not PhD thesis material but a language for a job.*¹

—James Gosling

That is, Java has always been an extremely practical language. Its attitude to performance was initially that, as long as the environment was *fast enough*, then raw performance could be sacrificed if developer productivity benefited. It was, therefore, not until 2005 or so, with the increasing maturity and sophistication of JVMs such as HotSpot, that the Java environment became suitable for high-performance computing applications.

This practicality manifests itself in many ways in the Java platform, but one of the most obvious is the use of *managed subsystems*. The idea is that the developer gives up some aspects of low-level control in exchange for

not having to worry about some of the details of the capability under management.

The most obvious example of this is, of course, memory management. The JVM provides automatic memory management in the form of a pluggable *garbage collection* subsystem (usually referred to as GC), so that memory does not have to be manually tracked by the programmer.

NOTE

Managed subsystems occur throughout the JVM, and their existence introduces extra complexity into the runtime behavior of JVM applications.

As we will discuss in the next section, the complex runtime behavior of JVM applications requires us to treat our applications as experiments under test. This leads us to think about the statistics of observed measurements, and here we make an unfortunate discovery.

The observed performance measurements of JVM applications are very often not normally distributed. This means that elementary statistical techniques (especially *standard deviation* and *variance*, for example) are ill-suited for handling results from JVM applications. This is because many basic statistics methods contain an implicit assumption about the normality of results distributions.

One way to understand this is that for JVM applications, outliers can be very significant—for a low-latency trading application or a ticket-booking system, for example. This means that sampling of measurements is also problematic, as it can easily miss the exact events that have the most importance.

Finally, a word of caution. It is very easy to be misled by Java performance measurements. The complexity of the environment means that it is very hard to isolate individual aspects of the system.

Measurement also has an overhead, and frequent sampling (or recording every result) can have an observable impact on the performance numbers being recorded. The nature of Java performance numbers requires a

certain amount of statistical sophistication, and naive techniques frequently produce incorrect results when applied to Java/JVM applications.

These concerns also resonate into the domain of cloud native applications. Automatic management of applications has very much become part of the cloud native experience—especially with the rise of *orchestration technologies* such as Kubernetes. The need to balance the cost of collecting data with the need to collect enough to make conclusions is also an important architectural concern for cloud native apps—we will have more to say about that in [Chapter 10](#).

Performance as an Experimental Science

The initial high-level picture you should have is that the JVM is a fast platform (and generally gets faster with each release), but despite that, Java applications can still be slow. This is because Java/JVM software stacks are, like most modern software systems, very complex.

In fact, due to the highly optimizing and adaptive nature of the JVM, production systems built on top of the JVM can have some subtle and intricate performance behavior. This complexity has been made possible by Moore’s law and the unprecedented growth in hardware capability that it represents.

The most amazing achievement of the computer software industry is its continuing cancellation of the steady and staggering gains made by the computer hardware industry.

—Henry Petroski (attr)

While some software systems have squandered the historical gains of the industry, the JVM represents something of an engineering triumph. Since its inception in the late 1990s, the JVM has developed into a very high-performance, general-purpose execution environment that puts those gains to very good use.

The tradeoff, however, is that like any complex, high-performance system, the JVM requires a measure of skill and experience to get the abso-

lute best out of it.

*A measurement not clearly defined is worse than useless.*²

—Eli Goldratt

JVM performance tuning is, therefore, a synthesis among technology, methodology, measurable quantities, and tools. Its aim is to effect measurable outputs in a manner desired by the owners or users of a system. In other words, performance is an experimental science—it achieves a desired result by:

1. Defining the desired outcome
2. Measuring the existing system
3. Determining what is to be done to achieve the requirement
4. Undertaking an improvement exercise
5. Retesting
6. Determining whether the goal has been achieved

The process of defining and determining desired performance outcomes builds a set of quantitative objectives. It is important to establish what should be measured and record the objectives, which then form part of the project's artifacts and deliverables. From this, we can see that performance analysis is based upon defining, and then achieving, nonfunctional requirements.

This process is, as has been previewed, not one of interpreting mysterious portents. Instead, we rely upon statistics and an appropriate handling (and interpretation) of results.

In this chapter, we discuss these techniques as they apply to a single JVM. In [Chapter 2](#), we will introduce a primer on the basic statistical techniques required for accurate handling of data generated from a JVM performance analysis project. Later on, primarily in [Chapter 10](#), we will discuss how these techniques generalize to a clustered application and give rise to the notion of observability.

It is important to recognize that, for many real-world projects, a more sophisticated understanding of data and statistics will undoubtedly be re-

quired. You are, therefore, encouraged to view the statistical techniques found in this book as a starting point, rather than a definitive statement.

A Taxonomy for Performance

In this section, we introduce some basic observable quantities for performance analysis. These provide a vocabulary for performance analysis and will allow you to frame the objectives of a tuning project in quantitative terms. These objectives are the nonfunctional requirements that define performance goals. Note that these quantities are not necessarily directly available in all cases, and some may require some work to obtain from the raw numbers obtained from our system.

One common basic set of performance observables is:

- Throughput
- Latency
- Capacity
- Utilization
- Efficiency
- Scalability
- Degradation

We will briefly discuss each in turn. Note that for most performance projects, not every metric will be optimized simultaneously. The case of only a few metrics being improved in a single performance iteration is far more common, and this may be as many as can be tuned at once. In real-world projects, it may well be the case that optimizing one metric comes at the detriment of another metric or group of metrics.

Throughput

Throughput is a metric that represents the rate of work a system or subsystem can perform. This is usually expressed as number of units of work in some time period. For example, we might be interested in how many transactions per second a system can execute.

For the throughput number to be meaningful in a real performance exercise, it should include a description of the reference platform it was ob-

tained on. For example, the hardware spec, OS, and software stack are all relevant to throughput, as is whether the system under test is a single server or a cluster. In addition, transactions (or units of work) should be the same between tests. Essentially, we should seek to ensure that the workload for throughput tests is kept consistent between runs.

Performance metrics are sometimes explained via metaphors that evoke plumbing. If we adopt this viewpoint, then, if a water pipe can produce one hundred liters per second, then the volume produced in one second (one hundred liters) is the throughput. Note that this value is a function of the speed of the water and the cross-sectional area of the pipe.

Latency

To continue the metaphor of the previous section—latency is how long it takes a given liter to traverse the pipe. This is a function of both the length of the pipe and how quickly the water is moving through it. It is not, however, a function of the diameter of the pipe.

In software, latency is normally quoted as an end-to-end time—the time taken to process a single transaction and see a result. It is dependent on workload, so a common approach is to produce a graph showing latency as a function of increasing workload. We will see an example of this type of graph in [“Reading Performance Graphs”](#).

Capacity

The capacity is the amount of work parallelism a system possesses—that is, the number of units of work (e.g., transactions) that can be simultaneously ongoing in the system.

Capacity is obviously related to throughput, and we should expect that as the concurrent load on a system increases, throughput (and latency) will be affected. For this reason, capacity is usually quoted as the processing available at a given value of latency or throughput.

For example, if we had a large reservoir at the beginning of our pipe, that would increase our capacity but not our overall throughput.

Alternatively, if we had a very narrow ingress into our pipe, and the pipe

then opened out, the capacity would be small, because the ingress acts as a choke point.

Utilization

One of the most common performance analysis tasks is to achieve efficient use of a system's resources. Ideally, CPUs should be used for handling units of work rather than being idle (or spending time handling OS or other housekeeping tasks).

Depending on the workload, there can be a huge difference between the utilization levels of different resources. For example, a computation-intensive workload (such as graphics processing or encryption) may be running at close to 100% CPU but be using only a small percentage of available memory.

As well as CPU, other resource types—such as network, memory, and (sometimes) the storage I/O subsystem—are becoming important resources to manage in cloud native applications. For many applications, more memory than CPU is “wasted,” and for many microservices, network traffic has become the real bottleneck.

In the scenario where the water pipe has a narrow ingress, although most of the pipe has a large throughput, the overall utilization is low (so water levels in the pipe would be low) because of the capacity restriction represented by the restricted ingress.

Efficiency

Dividing the throughput of a system by the utilized resources gives a measure of the overall efficiency of the system. Intuitively, this makes sense, as requiring more resources to produce the same throughput is one useful definition of being less efficient.

It is also possible, when one is dealing with larger systems, to use a form of cost accounting to measure efficiency. If solution A has a total cost of ownership (TCO) twice that of solution B for the same throughput, then it is, clearly, half as efficient.

Scalability

The throughput or capacity of a system, of course, depends upon the resources available for processing. The scalability of a system or application can be defined in several ways—but a useful definition is the change in throughput as resources are added. The holy grail of system scalability is to have throughput change exactly in step with resources.

Consider a system based on a cluster of servers. If the cluster is expanded, for example, by doubling in size, then what throughput can be achieved? If the new cluster can handle twice the volume of transactions, then the system is exhibiting “perfect linear scaling.” This is very difficult to achieve in practice, especially over a wide range of possible loads.

System scalability depends upon a number of factors and is not normally a simple linear relationship. It is very common for a system to scale close to linearly for some range of resources, but then at higher loads to encounter some limitation that prevents perfect scaling.

Degradation

If we increase the load on a system, either by increasing the rate at which requests arrive or the size of the individual requests, then we may see a change in the observed latency and/or throughput.

Note that this change depends on utilization. If the system is underutilized, then there should be some slack before observables change, but if resources are fully utilized, then we would expect to see throughput stop increasing or latency increase. These changes are usually called the degradation of the system under additional load.

Degradation also depends on a system’s architecture robustness. For example, if the pipe was made out of the same material as children’s balloons, the degradation under load would be pretty catastrophic. Once the load increased past a certain level, throughput would go to zero.

On the other hand, a more robust system would show a more realistic degradation scenario. For example, leaks that spring and get worse as pressure increases, or requests that are rejected before going into the sys-

tem, are similar to having the tap on too hard and water splashing out and not getting into the pipe.

Correlations Between the Observables

The behavior of the various performance observables is usually connected in some manner. The details of this connection will depend upon whether the system is running at peak.

For example, in general, the utilization will change as the load on a system increases. However, if the system is underutilized, then increasing load may not appreciably increase utilization. Conversely, if the system is already stressed, then the effect of increasing load may be felt in another observable.

As another example, scalability and degradation both represent the change in behavior of a system as more load is added. For scalability, as the load is increased, so are available resources, and the central question is whether the system can use them. On the other hand, if load is added but additional resources are not provided, degradation of some performance observable (e.g., latency) is the expected outcome.

NOTE

In rare cases, additional load can cause counterintuitive results. For example, if the change in load causes some part of the system to switch to a more resource-intensive but higher-performance mode, then the overall effect can be to reduce latency, even though more requests are being received.

To take one example, in [Chapter 6](#) we will discuss HotSpot’s JIT compiler in detail. To be considered eligible for JIT compilation, a method has to be executed in interpreted mode “sufficiently frequently.” So it is possible at low load to have key methods stuck in interpreted mode, but for those to become eligible for compilation at higher loads due to increased calling frequency on the methods. This causes later calls to the same method to run much, much faster than earlier executions.

Different workloads can have very different characteristics. For example, a trade on the financial markets, viewed end to end, may have an execu-

tion time (i.e., latency) of hours or even days. However, millions of them may be in progress at a major bank at any given time. Thus, the capacity of the system is very large, but the latency is also large.

However, let's consider only a single subsystem within the bank. The matching of a buyer and a seller (which is essentially the parties agreeing on a price) is known as *order matching*. This individual subsystem may have only hundreds of pending orders at any given time, but the latency from order acceptance to completed match may be as little as one millisecond (or even less in the case of “low-latency” trading).

In this section, we have met the most frequently encountered performance observables. Occasionally slightly different definitions, or even different metrics, are used, but in most cases these will be the basic system numbers that will normally be used to guide performance tuning and act as a taxonomy for discussing the performance of systems of interest.

Reading Performance Graphs

To conclude this chapter, let's look at some common patterns of behavior that occur in performance tests. We will explore these by looking at graphs of real observables, and we will encounter many other examples of graphs of our data as we proceed.

The graph in [Figure 1-1](#) shows sudden, unexpected degradation of performance (in this case, latency) under increasing load—commonly called a *performance elbow*.

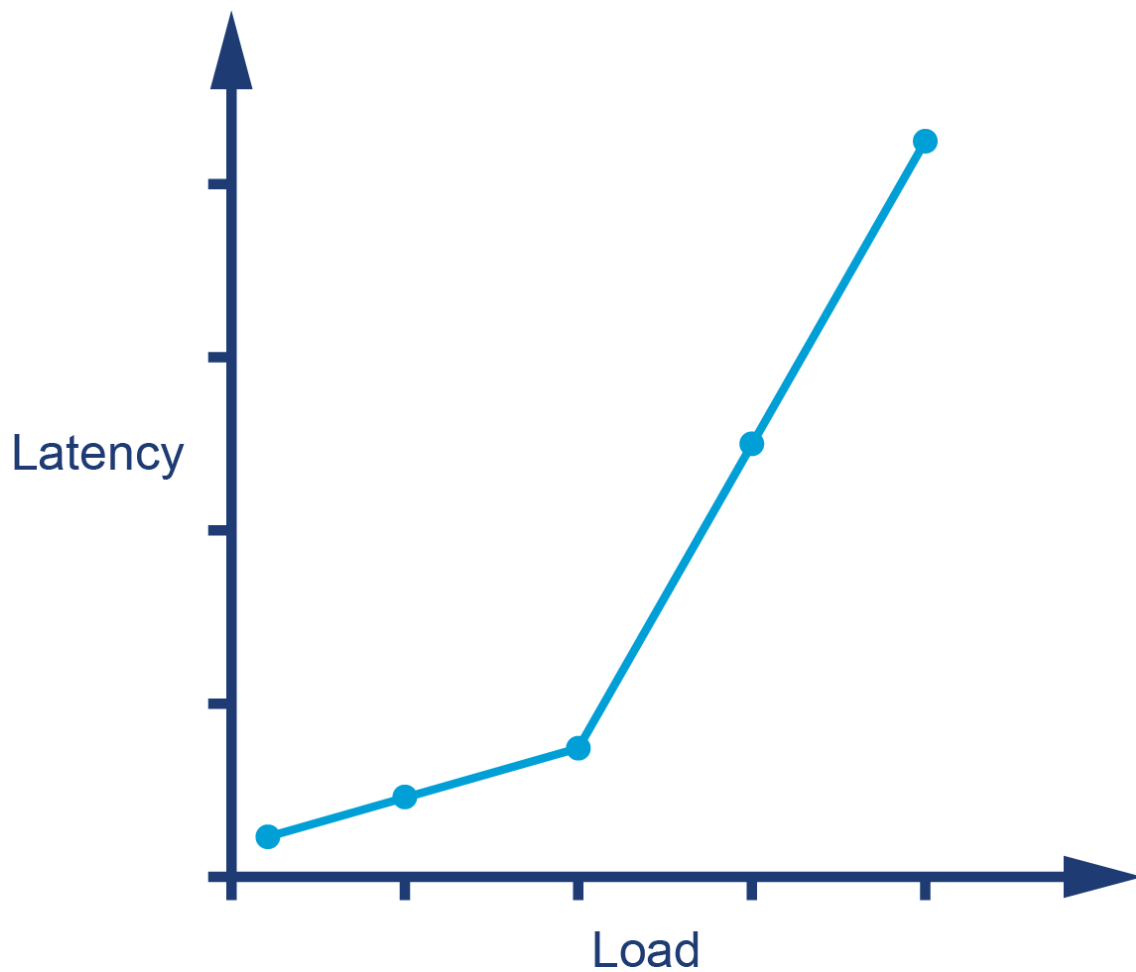


Figure 1-1. A performance elbow

By contrast, [Figure 1-2](#) shows the much happier case of throughput scaling almost linearly as machines are added to a cluster. This is close to ideal behavior and is only likely to be achieved in extremely favorable circumstances—e.g., scaling a stateless protocol with no need for session affinity with a single server.

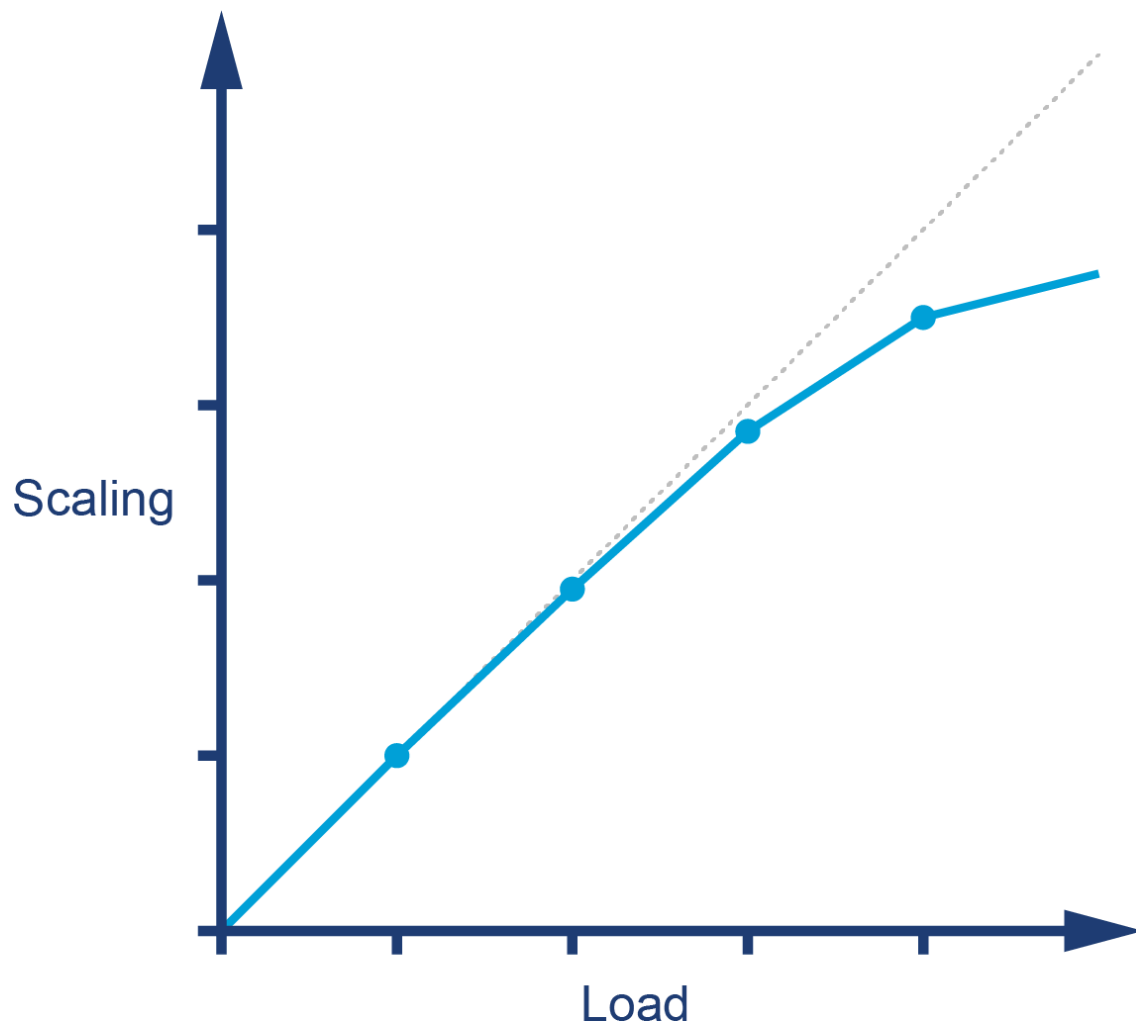


Figure 1-2. Near-linear scaling

In [Chapter 13](#) we will meet Amdahl’s law, named for the famous computer scientist (and “father of the mainframe”) Gene Amdahl of IBM.

[Figure 1-3](#) shows a graphical representation of his fundamental constraint on scalability: the maximum possible speedup as a function of the number of processors devoted to the task.

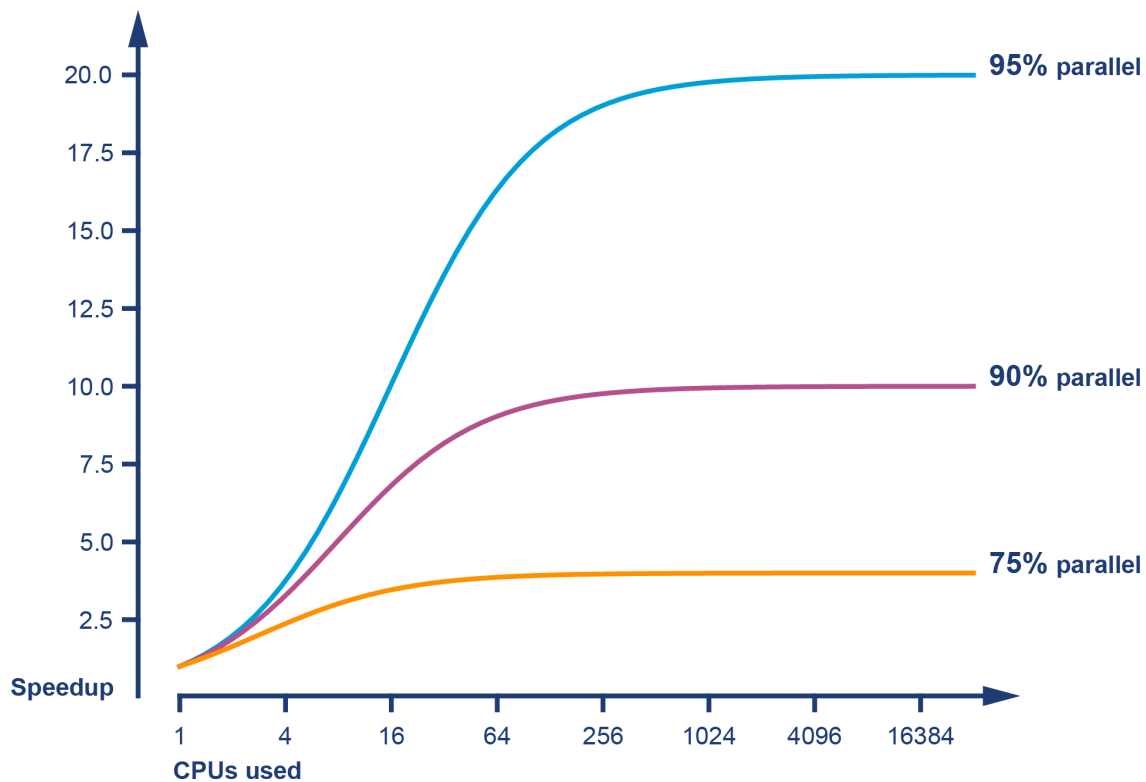


Figure 1-3. Amdahl's law

We display three cases: where the underlying task is 75%, 90%, and 95% parallelizable. This clearly shows that whenever the workload has any piece at all that must be performed serially, linear scalability is impossible, and there are strict limits on how much scalability can be achieved. This justifies the commentary around [Figure 1-2](#)—even in the best cases, linear scalability is all but impossible to achieve.

The limits imposed by Amdahl's law are surprisingly restrictive. Note in particular that the x-axis of the graph is logarithmic, so even with an algorithm that is 95% parallelizable (and thus only 5% serial), 32 processors are needed for a factor-of-12 speedup. Even worse, no matter how many cores are used, the maximum speedup is only a factor of 20 for that algorithm. In practice, many algorithms are far more than 5% serial, so they have a more constrained maximum possible speedup.

Another common subject of performance graphs in software systems is memory utilization. As we will see in [Chapter 4](#), the underlying technology in the JVM's garbage collection subsystem naturally gives rise to a “sawtooth” pattern of memory used for healthy applications that aren't under stress. We can see an example in [Figure 1-4](#), which is a close-up of a screenshot from the JDK Mission Control tool (JMC) provided by Eclipse Adoptium.

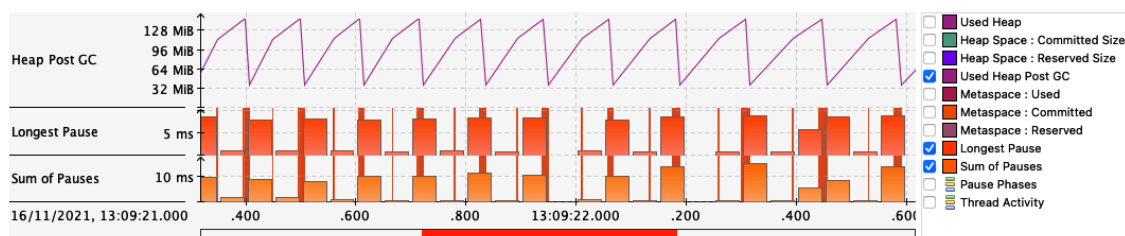


Figure 1-4. Healthy memory usage

One key performance metric for JVM is the allocation rate—effectively, how quickly it can create new objects (in bytes per second). We will have a great deal to say about this aspect of JVM performance in Chapters [4](#) and [5](#).

In [Figure 1-5](#), we can see a zoomed-in view of allocation rate, also captured from JMC. This has been generated from a benchmark program that is deliberately stressing the JVM’s memory subsystem—we have tried to make the JVM achieve 8 GiB/s of allocation, but as we can see, this is beyond the capability of the hardware, and instead the maximum allocation rate of the system is between 4 and 5 GiB/s.

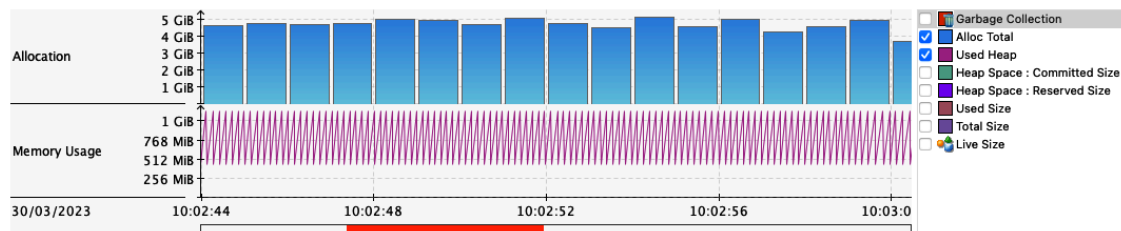


Figure 1-5. Sample problematic allocation rate

Note that tapped-out allocation is a different problem than the system having a resource leak. In that case, it is common for it to manifest in a manner like that shown in [Figure 1-6](#), where an observable (in this case latency) slowly degrades as the load is ramped up, before hitting an inflection point where the system rapidly degrades.

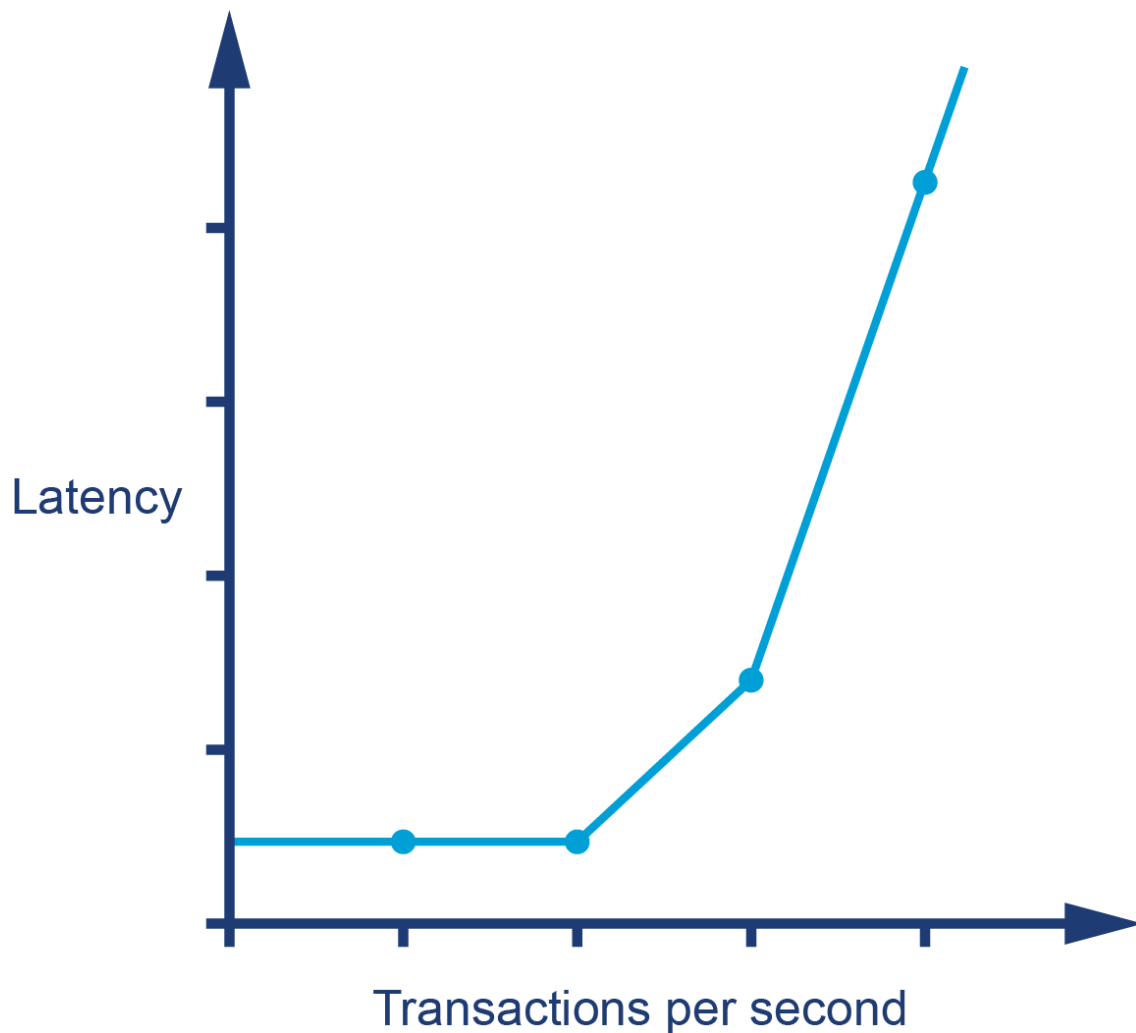


Figure 1-6. Degrading latency under higher load

Let's move on to discuss some extra things to consider when working with cloud systems.

Performance in Cloud Systems

Modern cloud systems are nearly always distributed systems in that they are composed of a cluster of nodes (JVM instances) interoperating via shared network resources. This means that, in addition to all the complexity of single-node systems, there is another level of complexity that must be addressed.

Operators of distributed systems have to think about things such as:

- How is work shared out among the nodes in the cluster?
- How do we roll out a new version of the software (or new config) to the cluster?
- What happens when a node leaves the cluster?
- What happens when a new node joins the cluster?

- What happens if the new node is misconfigured in some way?
- What happens if the new node behaves differently than the rest of the cluster in some way?
- What happens if there is a problem with the code that controls the cluster itself?
- What happens if there is a catastrophic failure of the entire cluster, or some infrastructure that it depends upon?
- What happens if a component in the infrastructure the cluster depends upon is a limited resource and becomes a bottleneck to scalability?

These concerns, which we will explore fully later in the book, have a major impact on how cloud systems behave. They affect the key performance observables such as throughput, latency, efficiency, and utilization.

Not only that, but there are two very important aspects—which differ from the single-JVM case—that may not be obvious at first sight to newcomers to cloud systems.

The first is that the unit of deployable code in the cloud is the *container* (we will have a great deal more to say about this later), rather than the application's JVM process (as it was in the days of big, bare-metal servers). It is also true that many possible performance impacts are caused by the internal behavior of a cluster, which may be opaque to the engineer.

We will discuss this in detail in [Chapter 10](#) when we tackle the topic of observability in modern systems and how to implement solutions to this visibility problem.

The second is that the efficiency and utilization of how a service uses cloud providers has a direct effect on the cost of running that service. Inefficiencies and misconfigurations can show up in the cost base of a service far more directly. In fact, this is one way to think about the rise of cloud.

In the old days, teams would often own actual physical servers in dedicated areas (usually called *cages*) in datacenters. Purchasing these servers represented *capital expenditure*, and the servers were tracked as an asset.

When we use cloud providers, such as AWS or Azure, we are renting time on machines actually owned by companies such as Amazon or Microsoft. This is *operational expenditure*, and it is a cost (or liability). This shift means that the computational requirements of our systems are now much more open to scrutiny by the financial folks.

Overall, it is important to recognize that cloud systems fundamentally consist of clusters of processes (in our case, JVMs) that dynamically change over time. The clusters can grow or shrink in size, but even if they do not, over time the participating processes will change. This stands in sharp contrast to traditional host-based systems, where the processes forming a cluster are usually much more long-lived and belong to a known—and stable—collection of hosts.

Summary

In this chapter, we have started to discuss what Java performance is and is not. We have introduced the fundamental topics of empirical science and measurement, and the basic vocabulary and observables that a good performance exercise will use. We have introduced some common cases that are often seen within the results obtained from performance tests. Finally, we have introduced the very basics of the sorts of additional issues that can arise in cloud systems.

Let's move on and begin discussing some major aspects of performance testing as well as how to handle the numbers that are generated by those tests.

¹ J. Gosling, “The Feel of Java,” *Computer*, vol. 30, no. 6 (June 1997): 53–57.

² Eliyahu Goldratt and Jeff Cox, *The Goal* (Gower Publishing, 1984).